VectorShip MVP — Product Requirements Document

Author: Product Lead **Audience:** Head of Engineering / Tech Lead **Version:** 1.0 — MVP Scope **Date:** March 2026

---

0. How to Read This Document

This document describes every screen, interaction, data structure, and system behavior for the VectorShip MVP. It is organized to mirror the user journey: intake → AI orchestration → generation → canvas composition → export. Each section contains the **what** (feature description), the **why** (product rationale), the **exactly how** (interaction specification), and the **boundaries** (what we are NOT building in MVP). JSON structures throughout are the canonical data contracts between frontend and backend. Wherever you see a `MVP_BOUNDARY` callout, that is a hard scope cut — do not architect for the excluded feature, only ensure the data model doesn't prevent it later.

---

1. Product Overview

VectorShip is a web application that generates production-ready vector logos and icons for indie hackers and small product teams. The user describes their product and visual preferences through a guided intake flow. An AI agent interprets their intent, generates structured prompts, and calls the Recraft API to produce transparent-background SVG elements. The user composes these elements on a canvas (positioning a mascot over a gradient background, for example), then exports a production-ready package for iOS, Android, and web — all without opening Figma or Illustrator.

The MVP delivers the complete loop: intake → generate → compose → export. It is a finished, polished product with attention to detail, not a prototype. However, it is scoped to logo/icon creation only — icon pack generation, mascot pose variations, and team features are post-MVP.

---

2. System Architecture Summary

Three systems collaborate, each with a strictly defined role.

System A — The Agent (LangGraph.js / ReAct). This is a server-side LangGraph.js agent running a ReAct (Reasoning + Acting) loop. It receives the user's raw input (text prompt, reference images, preference selections) and reasons through what to generate. It produces structured Prompt Specification Objects — JSON documents that describe exactly what Recraft should render. It maintains conversation state across the session, enabling iterative refinement ("make it rounder," "try a fox instead"). It persists session state to the database after every interaction. The agent has access to two tools: a `generate_element` tool that calls the Recraft

API, and a `analyze_image` tool that uses OpenAI Vision to extract style attributes from reference images.

System B — The Renderer (Recraft API). A stateless external API. Receives structured generation requests (prompt text, style parameters, color constraints, output format). Returns SVG data. Has no knowledge of the session, the user, or the canvas. Every call is independent.

System C — The Canvas (Client-Side React/SVG). A browser-based composition engine. Receives generated SVG elements from the backend and presents them on an interactive canvas. Handles all direct manipulation (drag, resize, rotate, recolor, layer management). Manages canvas state locally with optimistic updates, syncing to the server for persistence. Handles all export rendering (composing the final image, rasterizing at multiple sizes, packaging for download).

The critical architectural principle is that **these three systems communicate only through well-defined data contracts** (defined below). The agent never manipulates SVG DOM. The canvas never calls the Recraft API directly. The agent never renders pixels.

---

3. The Intake Flow — Screens & Interactions

The intake flow is a stepped wizard — three screens, each collecting a specific category of information. The user progresses linearly (Step 1 → 2 → 3) but can navigate back to any previous step. All collected data is accumulated in a single `IntakePayload` object that is sent to the agent upon completion.

3.1 Screen 1: "Tell Us About Your Product"

Purpose: Collect the raw creative brief — what the user is building, who it's for, and what feeling they want to convey.

Layout: A clean, single-column form centered on the page. No sidebar, no canvas visible yet. The tone is conversational, not clinical.

Fields:

Field 1 — Product Description (required). A single multi-line text input with the placeholder: "Describe your product in a few sentences. What does it do? What makes it special?" This is the primary creative seed. The agent will extract the product name, industry vertical, audience, and emotional tone from this text. Minimum 20 characters, maximum 500 characters. Display a soft character counter in the bottom-right of the input (e.g., "47 / 500"). No hard validation — if the user types less than 20 characters, show a gentle nudge below the field: "A bit more detail helps us generate better results."

Field 2 — Reference Images (optional). A drag-and-drop zone that accepts up to 3 images (PNG, JPG, WEBP; max 5MB each). The zone shows a dashed border with the text "Drop reference images here, or click to browse" and a subtle icon. When images are uploaded, they display as thumbnails (80×80px, cover-fit) in a horizontal row below the drop zone, each with an × button

to remove. Below the thumbnails, a secondary text input appears: "What do you like about these references?" (optional, max 200 characters). This gives the agent additional context for style analysis.

Field 3 — Brand Personality (required, min 1, max 4). A horizontally-wrapped grid of pill-shaped toggles. Each pill contains a single word. The user taps to select (pill fills with the brand accent color) and taps again to deselect. Selected pills are visually distinct (filled background, white text). Maximum 4 selections enforced — if the user taps a 5th, the earliest selection is not auto-deselected; instead, the 5th pill briefly shakes and a tooltip appears: "Pick up to 4." The curated list, in this exact display order:

`Playful` · `Minimal` · `Bold` · `Elegant` · `Technical` · `Friendly` · `Serious` · `Quirky` · `Premium` · `Approachable` · `Futuristic` · `Retro` · `Organic` · `Geometric`

These 14 words were chosen because they map cleanly to visual attributes. The agent has a mapping table (defined in Section 4) that translates each word into Recraft style modifiers.

Navigation: A single "Continue" button at the bottom-right, enabled only when Field 1 has ⩾20 characters and Field 3 has ⩾1 selection. Clicking "Continue" does NOT trigger any API call — it simply advances to Screen 2. All data stays client-side until the full intake is submitted.

3.2 Screen 2: "Choose Your Style Direction"

Purpose: Collect visual style preference through visual selection (not text), and color preference.

Layout: Two sections stacked vertically on the same screen.

Section A — Style Grid. A responsive grid of 8 style cluster cards. Each card is approximately 160×160px, containing a curated reference image (a real, hand-picked example of that style, similar to the app icon grids from the user's reference Images 3 and 4) and a label below it. The cards are arranged in a 4×2 grid on desktop, 2×4 on mobile. The user can select exactly 1 card (single-select radio behavior). Selected card gets a prominent border and a small checkmark badge in the corner.

The 8 style clusters, with their labels and descriptions (description shown as a tooltip on hover, not permanently visible):

Cluster ID	Label	Tooltip Description
`flat_mascot`	Flat Mascot	Cute character with simple shapes, dot eyes, minimal detail
`gradient_abstract`	Gradient Abstract	Bold gradients with abstract organic or geometric shapes
`line_mark`	Line Mark	Clean single-weight line art, monochrome or two-tone
`3d_rendered`	3D Rendered	Soft 3D-style with subtle depth, highlights, and rounded forms
`minimal_geometric`	Minimal Geometric	Pure geometric shapes, very few elements, mathematical precision
`playful_character`	Playful Character	Expressive character with personality, more detail than flat mascot
`typographic`	Typographic	Letterform-centric, the product name IS the icon
`icon_centric`	Icon-Centric	A single recognizable symbol or object, clean and bold

Each card image is a static asset — not AI-generated. These are curated examples that represent the style cluster. They should be high-quality screenshots or compositions of 4-6 real app icons in that style, similar to the reference grids the user provided. These images are stored as static assets in the project and do not change.

`MVP_BOUNDARY:` In MVP, the user selects exactly 1 style cluster. Post-MVP, we may allow selecting 2 and blending them. The data model supports this (array field) but the UI enforces single selection.

Section B — Color Preference. Three horizontally-arranged option cards (not a dropdown — visible choices). The user selects exactly 1:

Option 1: "I have brand colors." When selected, a color input section expands below with 3 color slots: Primary, Secondary, and Accent. Each slot is a circular color swatch (32px diameter) that opens a standard color picker on click, plus a text input for hex code entry beside it. Primary is required; Secondary and Accent are optional. Default values: Primary `#6366F1` (indigo), Secondary `#EC4899` (pink), Accent `#F59E0B` (amber) — these are pre-filled as suggestions but editable.

Option 2: "Suggest colors for my vibe." When selected, the section below shows a brief text: "We'll generate a palette based on your brand personality." No additional input needed. The agent will use the personality words from Screen 1 to generate a color palette.

Option 3: "I'll decide later." When selected, the section below shows: "We'll generate in neutral tones. You can recolor everything on the canvas." No additional input needed.

Navigation: "Back" button (top-left, text link style) returns to Screen 1 with all data preserved. "Generate My Logo" button (bottom-right, primary action style — filled, prominent) submits the full intake and triggers the AI pipeline. This button shows a loading state on click (spinner replaces text, button becomes non-interactive).

3.3 Intake Data Contract

When the user clicks "Generate My Logo," the client assembles and sends the following payload to the backend:

```json
{
  "product_description": "Taskpaw is a task management app for pet owners who want t
  "reference_images": [
    {
      "file_id": "ref_001",
      "mime_type": "image/png",
      "base64": "...",
      "user_note": "I love the rounded style and the simple face"
    }
  ],
  "personality_words": ["playful", "friendly", "approachable"],
  "style_cluster": "flat_mascot",
  "color_preference": {
    "mode": "custom",
    "colors": {
      "primary": "#F97316",
      "secondary": "#EC4899",
      "accent": null
    }
  }
}
```

Alternative `color_preference` values:

```json
{ "mode": "auto_suggest", "colors": null }
{ "mode": "neutral", "colors": null }
```

This payload is the sole input to the agent. It contains everything needed to begin generation. The `reference_images` array can be empty (user uploaded nothing). The `personality_words`

array contains 1-4 strings from the curated list. The `style_cluster` is one of the 8 cluster IDs. This contract is stable — the agent must accept any valid combination of these fields.

---

4. The AI Agent — LangGraph.js / ReAct Architecture

4.1 Agent Design Philosophy

The agent is NOT a simple prompt-in, prompt-out wrapper. It is a reasoning agent that makes multi-step decisions about what to generate, analyzes intermediate results, and adapts its strategy based on user feedback. We use LangGraph.js with ReAct methodology because the generation task genuinely requires reasoning: "The user said 'playful' but uploaded a reference image that's quite minimal — I should lean more minimal-playful than cartoon-playful."

The agent runs server-side in a Next.js API route (or a dedicated worker). It is invoked by HTTP request and returns structured results. It is NOT a streaming chat interface — the user sees a loading screen while the agent works, then receives all results at once.

4.2 Agent State

The agent maintains a state object that is persisted to the database after every invocation. This state is the single source of truth for the session.

```json
```

```
{
  "session_id": "sess_a1b2c3d4",
  "user_id": "usr_x1y2z3",
  "created_at": "2026-03-21T10:00:00Z",
  "updated_at": "2026-03-21T10:05:30Z",
  "status": "elements_generated",

  "intake": {
    "product_description": "...",
    "reference_images": ["ref_001"],
    "personality_words": ["playful", "friendly", "approachable"],
    "style_cluster": "flat_mascot",
    "color_preference": { "mode": "custom", "colors": { "primary": "#F97316", "secon
  },

  "reference_analysis": {
    "ref_001": {
      "style_attributes": ["rounded shapes", "simple dot eyes", "white body", "minim
      "color_attributes": ["warm gradient background", "white foreground", "pink-to-
      "complexity": "low"
    }
  },

  "derived_style": {
    "recraft_style_preset": "digital_illustration",
    "stroke_weight": "none",
    "shape_language": "rounded, organic, soft edges",
    "detail_level": "low",
    "color_palette": {
      "primary": "#F97316",
      "secondary": "#EC4899",
      "background_gradient": { "from": "#F97316", "to": "#EC4899", "angle": 135 },
      "foreground_primary": "#FFFFFF",
      "foreground_accent": "#1A1A1A"
    },
    "visual_motifs": ["cat", "paw", "pet-related"],
    "avoid": ["complex textures", "photorealistic shading", "thin lines"]
  },

  "generation_rounds": [
    {
      "round_number": 1,
      "trigger": "initial_generation",
      "elements_requested": [
        {
          "element_id": "el_gen_001",
```

```
          "element_type": "primary_mark",
          "recraft_prompt": "A cute simple cat character, minimal flat design, round
          "recraft_params": {
            "style": "digital_illustration",
            "substyle": "flat_2",
            "model": "recraftv3",
            "size": "1024x1024",
            "response_format": "svg"
          },
          "status": "completed",
          "result_svg_storage_key": "sessions/sess_a1b2c3d4/elements/el_gen_001.svg"
          "result_thumbnail_url": "/api/thumbnails/el_gen_001.png"
        }
      ],
      "user_feedback": null
    }
  ],

  "canvas_state": null
}
```


4.3 Agent Tools

The agent has access to exactly two tools in MVP.

Tool 1: `analyze_reference_image` Called once per uploaded reference image during the initial processing step. Sends the image to OpenAI GPT-4o Vision with a structured analysis prompt requesting: style attributes (list of 3-7 adjectives), color attributes (list of 2-4 descriptions), composition attributes (centered, asymmetric, etc.), and complexity level (low, medium, high). The response is parsed into the `reference_analysis` field of the session state. This tool is called BEFORE any Recraft generation — its output informs the prompt construction.

Tool 2: `generate_element` Called to generate a single SVG element via the Recraft API. Accepts: prompt text (string), Recraft style parameters (object), and an element type label (string, one of: `primary_mark`, `secondary_mark`, `accent_element`, `background_shape`, `text_element`). Returns: the SVG string and a storage key where the SVG has been saved. The agent calls this tool multiple times in a single invocation to generate a batch of elements.

4.4 Agent Reasoning Flow — Initial Generation

When the agent receives the intake payload for a new session, it executes the following reasoning steps (this is the ReAct loop):

Step 1 — Observation. "I have a product description, personality words, a style cluster selection, and possibly reference images. Let me analyze the references first."

Action: If reference images exist, call `analyze_reference_image` for each one. Store results in `reference_analysis`.

Step 2 — Reasoning. "Based on the product description, personality words, style cluster, reference analysis, and color preferences, I need to construct a unified `derived_style` object that captures the complete visual direction."

Action: The agent uses its system prompt (which contains the personality-to-style mapping table, detailed below) and the reference analysis to construct `derived_style`. This is a pure reasoning step — no tool call, just LLM inference that populates the style object.

Personality-to-Style Mapping Table (included in the agent's system prompt):

```json
{
  "playful": { "shape_bias": "rounded, organic", "color_bias": "warm, saturated", "d
  "minimal": { "shape_bias": "geometric, clean", "color_bias": "monochrome or 2-colo
  "bold": { "shape_bias": "large, thick, strong", "color_bias": "high contrast, vivi
  "elegant": { "shape_bias": "thin, refined, symmetrical", "color_bias": "muted, sop
  "technical": { "shape_bias": "angular, precise, grid-aligned", "color_bias": "cool
  "friendly": { "shape_bias": "rounded, soft, approachable", "color_bias": "warm, in
  "serious": { "shape_bias": "structured, squared, stable", "color_bias": "dark, pro
  "quirky": { "shape_bias": "irregular, unexpected, asymmetric", "color_bias": "unus
  "premium": { "shape_bias": "refined, balanced, symmetrical", "color_bias": "gold,
  "approachable": { "shape_bias": "rounded, open, soft", "color_bias": "warm pastels
  "futuristic": { "shape_bias": "angular, dynamic, layered", "color_bias": "neon, gr
  "retro": { "shape_bias": "chunky, textured feel, rounded", "color_bias": "muted wa
  "organic": { "shape_bias": "flowing, natural curves, irregular", "color_bias": "ea
  "geometric": { "shape_bias": "precise, grid-based, mathematical", "color_bias": "a
}
```

The agent synthesizes the selected personality words by finding the intersection and weighting shared attributes. For example, "playful" + "friendly" + "approachable" all share "rounded" and "warm" — these become strong style signals. If conflicting words are selected ("minimal" + "bold"), the agent resolves by prioritizing the style cluster selection as the tiebreaker.

Step 3 — Planning. "Now I need to decide what elements to generate. Based on the style cluster and derived style, I'll create a generation plan."

The generation plan follows this template based on the style cluster:

Style Cluster	Elements Generated
`flat_mascot`	4× primary_mark (mascot character variations), 2× accent_element (small related icon)
`gradient_abstract`	4× primary_mark (abstract shape variations), 2× accent_element (smaller complementary shapes)
`line_mark`	4× primary_mark (line art mark variations), 2× accent_element (line detail elements)
`3d_rendered`	4× primary_mark (3D object/character variations), 1× accent_element
`minimal_geometric`	4× primary_mark (geometric mark variations), 2× accent_element (geometric accents)
`playful_character`	4× primary_mark (character variations with more detail), 2× accent_element
`typographic`	4× primary_mark (letterform/wordmark variations), 1× accent_element
`icon_centric`	4× primary_mark (icon symbol variations), 2× accent_element

Total elements per initial generation: 5-6. This is a hard limit for MVP. The agent MUST NOT generate more than 6 elements in the initial round (cost control). Each element is a separate Recraft API call.

Background shapes are NOT generated by the AI. Backgrounds (solid colors, gradients) are composed client-side on the canvas. The agent does, however, include a `suggested_backgrounds` field in its response with 3 gradient suggestions derived from the color palette.

Step 4 — Execution. Call `generate_element` for each planned element. The calls can be parallelized (Recraft's API supports concurrent requests). Wait for all results. Store each SVG to object storage (Supabase Storage or S3) and record the storage keys in the session state.

Step 5 — Response Assembly. Package the results into the `GenerationResponse` object (defined below) and return it to the frontend.

4.5 Agent Reasoning Flow — Regeneration / Iteration

When the user requests changes (from the canvas screen), the frontend sends a `RegenerationRequest`:

```
json
```

```
{
  "session_id": "sess_a1b2c3d4",
  "instruction": "Make the cat rounder with bigger eyes, and try one with a dog inst
  "keep_elements": ["el_gen_003"],
  "discard_elements": ["el_gen_001", "el_gen_002"]
}
```

The agent loads the full session state, appends this as a new generation round, reasons about how to modify the prompts (incorporating the instruction into the prompt text, keeping the derived_style consistent), and generates new elements. The instruction is free-text — the agent interprets it. The `keep_elements` and `discard_elements` arrays tell the agent which previous results the user liked (informing style direction) and which they didn't (informing what to avoid).

`MVP_BOUNDARY:` In MVP, regeneration replaces all discarded elements in a single batch. There is no per-element "regenerate just this one" button, though the user can keep specific elements and only regenerate the rest. Post-MVP, individual element regeneration and fine-grained prompt editing become available.

4.6 Generation Response Contract

The agent returns this to the frontend after each generation round:

```
json
```

```
{
  "session_id": "sess_a1b2c3d4",
  "round_number": 1,
  "elements": [
    {
      "element_id": "el_gen_001",
      "element_type": "primary_mark",
      "svg_url": "/api/elements/el_gen_001.svg",
      "thumbnail_url": "/api/thumbnails/el_gen_001.png",
      "detected_colors": ["#FFFFFF", "#1A1A1A", "#F97316"],
      "svg_dimensions": { "width": 1024, "height": 1024 },
      "description": "Simple white cat character with orange ear accents, dot eyes,
    },
    {
      "element_id": "el_gen_002",
      "element_type": "primary_mark",
      "svg_url": "/api/elements/el_gen_002.svg",
      "thumbnail_url": "/api/thumbnails/el_gen_002.png",
      "detected_colors": ["#FFFFFF", "#1A1A1A", "#EC4899"],
      "description": "Simple white cat with pink bow, slightly tilted head"
    }
  ],
  "suggested_backgrounds": [
    { "type": "linear_gradient", "from": "#F97316", "to": "#EC4899", "angle": 135 },
    { "type": "linear_gradient", "from": "#F97316", "to": "#FBBF24", "angle": 180 },
    { "type": "solid", "color": "#1A1A1A" }
  ],
  "suggested_canvas_setup": {
    "container_shape": "ios_squircle",
    "recommended_element_placement": {
      "primary_mark": { "x_percent": 50, "y_percent": 50, "scale_percent": 70 },
      "accent_element": { "x_percent": 75, "y_percent": 25, "scale_percent": 15 }
    }
  }
}
```


The `detected_colors` array is computed server-side by parsing the SVG and extracting unique fill/stroke hex values. This powers the recolor panel on the canvas. The `suggested_canvas_setup` provides intelligent defaults so the first canvas render already looks composed, not scattered.

---

5. The Loading / Generation Screen

Purpose: Bridge the 10-25 second gap between intake submission and generation completion. This screen must feel fast and intentional, not like a broken loading state.

Layout: Full-page, centered content. No navigation, no sidebar.

Content sequence: A centered animation area (200×200px) showing a subtle, looping animation (abstract morphing shapes, NOT a spinner). Below it, a text line that cycles through generation-stage messages every 3-4 seconds:

1. "Understanding your brand personality..."
2. "Analyzing your style references..." (only if references were uploaded; skip otherwise)
3. "Crafting your icon elements..."
4. "Generating variations..."
5. "Finalizing your options..."

These messages are timed on the frontend, not driven by actual backend progress. The animation runs until the generation response arrives, then the screen transitions to the canvas.

Error handling: If the generation takes longer than 45 seconds, show a subtle secondary message: "This is taking a bit longer than usual. Hang tight." If it exceeds 90 seconds, show: "Something may have gone wrong. [Try again]" — the "Try again" button resubmits the same intake payload.

Transition: When the response arrives, the loading screen fades out (300ms opacity transition) and the canvas screen fades in. The generated elements should already be positioned on the canvas according to the `suggested_canvas_setup` from the response, so the user sees a composed result immediately — not an empty canvas with a sidebar of unplaced elements.

---

6. The Canvas — Complete Specification

This is the core of the product and the most complex frontend surface. It has four distinct zones: the Canvas Area (center), the Toolbar (top), the Element Gallery (left sidebar), and the Properties Panel (right sidebar).

6.1 Overall Layout

Desktop (≥1024px): Three-column layout. Left sidebar (260px fixed width) contains the Element Gallery. Center area (flexible, fills remaining space) contains the Canvas. Right sidebar (280px fixed width) contains the Properties Panel. Top bar (56px height, full width) contains the Toolbar. The sidebars are always visible on desktop.

Tablet (768–1023px): The left sidebar collapses to an icon-only rail (48px). Clicking a rail icon opens the sidebar as an overlay (280px) on top of the canvas. The right sidebar becomes a slide-

out panel triggered by selecting an element on the canvas. The canvas area fills the remaining space.

Mobile (< 768px): `MVP_BOUNDARY:` Mobile canvas editing is out of scope for MVP. On mobile viewports, show a "Best experienced on desktop" message with the option to view a read-only preview of the canvas composition and trigger exports. Full mobile editing is post-MVP.

6.2 The Canvas Area

The canvas area is the centerpiece. It displays a composition surface with the following layers, from back to front:

Layer 0 — The Page Background. A neutral checkerboard pattern (light gray / white squares, 16px each) indicating transparency. This is not part of the export — it's purely visual feedback showing "there's nothing here." This pattern fills the entire canvas area behind everything else.

Layer 1 — The Container Shape. The selected container shape (iOS squircle, Android adaptive circle, rounded rectangle, circle, or none/freeform), rendered at the center of the canvas area. This shape defines the "downloadable zone" — everything inside it will be exported, everything outside will be clipped. The container is rendered with a subtle dashed border (1px, medium gray) when the container type is not "none." Inside the container, the background fill (solid color, linear gradient, or radial gradient) is rendered. When the container type is "none," the container is a simple rectangle matching the canvas dimensions, with no visible border.

The container is always centered in the canvas area. Its rendered size adapts to the canvas area dimensions, maintaining a 1:1 aspect ratio and fitting within the available space with at least 40px of padding on all sides. The actual export resolution is always 1024×1024 — the canvas display is a scaled representation.

Layer 2+ — Generated Elements. Each SVG element from the generation response is rendered as an independent layer on top of the container. Elements are positioned, scaled, and rotated according to their transform properties. Elements extend beyond the container boundary during editing (they're not clipped while editing), but a semi-transparent overlay outside the container boundary provides visual feedback about what will and won't be included in the export.

Canvas Interactions:

Pan: Click and drag on the empty canvas area (not on an element) to pan the view. Scroll wheel zooms in/out. The zoom range is 25% to 400%, with 100% being the default (container fills the available space comfortably). Zoom percentage is shown in the bottom-right corner of the canvas area as a small badge, clickable to reset to 100%.

Select: Click on an element to select it. The selected element shows 8 resize handles (4 corners + 4 edge midpoints), a rotation handle (circular, positioned 20px above the top-center), and a bounding box (thin blue border, 1px). Only one element can be selected at a time in MVP. Clicking on empty canvas deselects.

Drag to Move: Click and drag a selected element to reposition it. While dragging, show alignment guides: a vertical center line and horizontal center line of the container, and edge-

snapping guides when the element's edge approaches the container's edge or center (within 8px snap threshold). The guides are thin (1px) magenta lines that appear only when the element is within snap range. Snapping is magnetic — the element subtly jumps to the guide position when within threshold.

Resize: Drag any of the 8 handles to resize the element. Corner handles resize proportionally by default (maintaining aspect ratio). Holding Shift while dragging a corner handle enables free (non-proportional) resize. Edge midpoint handles resize in a single axis only. During resize, the element scales from the opposite handle as the anchor point. Show the current dimensions (width × height in px) as a small tooltip near the cursor during resize.

Rotate: Drag the rotation handle (the circle above the element). Rotation is continuous (0-360°). Holding Shift while rotating snaps to 15° increments. Show the current rotation angle as a tooltip during rotation.

Deselect: Click on empty canvas, or press Escape.

Delete: Press Delete or Backspace when an element is selected. The element is removed from the canvas but NOT deleted from the session — it moves back to the Element Gallery (left sidebar) in a "removed" state, so the user can re-add it.

6.3 The Toolbar (Top Bar)

The toolbar spans the full width of the screen, height 56px. It contains, from left to right:

Logo / Home Link. The VectorShip wordmark/logo, clicking returns to the dashboard (with a confirmation dialog if there are unsaved changes).

Session Title. An editable text field showing the session name (defaults to "Untitled Logo" — clicking allows inline editing). Purely for the user's organization. Saved to session state on blur.

Undo / Redo Buttons. Icon buttons (↶ / ↷). Undo reverses the last canvas action (move, resize, rotate, recolor, layer reorder, element add/remove, background change). Redo re-applies. The undo stack maintains the last 30 actions. Keyboard shortcuts: Cmd/Ctrl+Z for undo, Cmd/Ctrl+Shift+Z for redo.

Container Shape Dropdown. A compact dropdown selector showing the current container shape as an icon + label. Options: "iOS (Squircle)" / "Android (Circle)" / "Rounded Rect" / "Circle" / "None (Freeform)." Changing the container immediately updates the canvas display. When "Rounded Rect" is selected, an additional small numeric input appears beside the dropdown for the corner radius (range 0-50%, default 20%).

Grid Toggle. An icon button (grid icon) that toggles the pixel grid overlay on/off. When on, a subtle grid (16px spacing, very faint lines) overlays the canvas within the container boundary. Default state: off.

Safe Zone Toggle. An icon button (frame/margin icon) that toggles safe zone guides. When on, shows the appropriate safe zone for the selected container shape: for iOS, the safe zone is the area within the squircle after Apple's corner masking; for Android, it's the 66dp inner circle

(within the 108dp adaptive icon frame). Safe zones render as a thin dashed border inside the container. Default state: off.

Zoom Controls. "−" and "+" icon buttons for zoom, with the current percentage between them. Clicking the percentage resets to 100%.

Regenerate Button. A secondary-style button: "Regenerate." Clicking opens a small modal/popover with a text input ("What would you like to change?") and two checkbox lists: one for elements to keep (pre-checked) and one for elements to discard (unchecked). A "Generate New Options" button submits the `RegenerationRequest` to the agent and shows the loading screen again.

Export Button. A primary-style button: "Export." Clicking opens the Export Modal (defined in Section 7).

6.4 The Element Gallery (Left Sidebar)

Purpose: Show all generated elements from the current session, allow the user to add/remove them from the canvas, and provide element-level actions.

Layout: A vertical scrollable list of element cards. Each card is 220×160px (fitting the 260px sidebar with 20px padding on each side).

Card contents: The element's SVG rendered as a preview thumbnail on a light checkerboard background (showing transparency). Below the preview: the element's description text (truncated to 2 lines, from `description` in the generation response), and a subtle label showing the element type ("Primary Mark", "Accent").

Card states:

"On canvas" — The card has a colored left border (blue, 3px) and a small checkmark badge in the top-right corner. The element is currently placed on the canvas.

"Available" — The card has no colored border, no badge. The element exists in the generation results but is not currently on the canvas. Single-clicking the card adds it to the canvas at the `suggested_canvas_setup` position (or center if no suggestion exists for that element type).

"Loading" — A shimmer animation over the card during regeneration.

Drag from gallery: `MVP_BOUNDARY:` In MVP, clicking adds to center. Drag-from-sidebar-to-canvas is a post-MVP enhancement.

Section divider: If there are multiple generation rounds, a thin divider with the label "Round 2" separates the element groups.

6.5 The Properties Panel (Right Sidebar)

Purpose: Precise control over the selected element's properties and the canvas background.

The panel's content changes based on what's selected:

When no element is selected — Background Properties:

Background Type Selector. Three pill toggles: "Solid" / "Gradient" / "None." Default is "Gradient" using the first `suggested_backgrounds` from the generation response.

If Solid: A single color swatch + hex input for the background color.

If Gradient: Two color swatches + hex inputs (From and To colors), a circular angle control (a small circle with a draggable dot on its perimeter, showing the gradient angle 0-359°), and a toggle between "Linear" and "Radial." The gradient live-updates on the canvas as the user adjusts controls.

If None: The background is transparent (checkerboard visible). This is what the user selects if they want to export a transparent PNG or just the raw SVG elements.

Suggested Backgrounds: Below the manual controls, a row of 3 small preview squares (48×48px) showing the `suggested_backgrounds` from the generation response. Clicking one applies it immediately (populating the gradient/solid controls with its values).

When an element is selected — Element Properties:

Transform Section:

Position — Two numeric inputs labeled "X" and "Y" (in pixels, relative to the canvas container's top-left corner). Updating these moves the element. Step: 1px. Arrow keys nudge by 1px (10px with Shift held).

Scale — A single numeric input showing the element's scale as a percentage (100% = original size). Range: 10% to 500%. Alternatively, two linked width/height inputs that maintain aspect ratio (with a link/unlink toggle icon between them).

Rotation — A numeric input showing degrees (0-359). A small circular angle control beside it for direct manipulation.

Opacity — A horizontal slider from 0% to 100%, with numeric input beside it. Default: 100%.

Action Buttons:

"Flip Horizontal" (icon button) — Mirrors the element on its vertical axis.
"Flip Vertical" (icon button) — Mirrors the element on its horizontal axis.
"Duplicate" (icon button) — Creates a copy of the element, offset by (16px, 16px) from the original.
"Remove from Canvas" (icon button, red tint) — Removes the element from the canvas and returns it to the gallery in "Available" state.

Recolor Section:

Header: "Colors" with a small label "(click to change)."

Below the header, a horizontal-wrap grid of circular swatches (28px diameter each), one for each unique fill/stroke color detected in the SVG element. The swatches display the actual color.

Clicking a swatch opens a popover with a color picker (hue ring + saturation/brightness square + hex input + opacity slider). Changing the color immediately updates every instance of that color in the SVG element on the canvas. The replacement is implemented by traversing the SVG DOM and replacing all `fill` and `stroke` attribute values matching the original hex with the new hex. This is a global find-and-replace within that element's SVG tree.

Edge case: Some SVGs may contain colors that are visually very similar but technically different hex values (e.g., `#FFFFFF` and `#FEFEFE`). The color detection should cluster colors within a delta-E distance of 5 and present them as a single swatch, replacing all clustered variants when the user changes one.

Layer Order Section:

Header: "Layers."

A vertical list showing all elements currently on the canvas, from topmost to bottommost. Each list item shows a tiny thumbnail (32×32px) and the element description (truncated). The list supports drag-to-reorder (reordering the list changes the z-order on the canvas). Each list item has an eye icon (toggle visibility) and a lock icon (toggle position lock — prevents accidental moves). This section is always visible at the bottom of the Properties Panel, regardless of whether an element is selected. When an element is selected in the layers list, it becomes selected on the canvas as well (and vice versa).

---

7. The Export Pipeline

7.1 Export Modal

Triggered by the "Export" button in the toolbar. A centered modal dialog (560px wide, max-height 80vh, scrollable).

Modal Header: "Export Your Logo"

Modal Body — Export Target Selector. A vertical list of export targets, each as a selectable card. Multiple can be selected simultaneously:

Target 1: "iOS / macOS App Icon" Description text: "Xcode-ready asset catalog. All sizes from 20pt to 1024pt, @1x to @3x. Drop directly into your Xcode project." Details shown when selected: List of sizes to be generated (plain text, not editable — just for transparency). Output: `.zip` file containing `AppIcon.appiconset/` folder with all PNGs + `Contents.json`.

Target 2: "Android Adaptive Icon" Description text: "Foreground and background layers with mipmap folders. Drop into your Android Studio project." Details: Shows foreground/background layer separation preview (a small visual showing which canvas layers become foreground vs background). Output: `.zip` file containing `mipmap-mdpi/` through `mipmap-xxxhdpi/` + `ic_launcher.xml` + `ic_launcher_round.xml`.

Target 3: "Web / PWA Favicons" Description text: "Complete favicon set with site.webmanifest and HTML meta tags. Copy into your project root." Details: Shows list of generated files. Output: `.zip` file containing `favicon.ico`, `favicon-16x16.png`, `favicon-32x32.png`, `apple-touch-icon.png`, `android-chrome-192x192.png`, `android-chrome-512x512.png`, `site.webmanifest`, `meta-tags.html` (snippet).

Target 4: "OG / Social Image" Description text: "1200×630 Open Graph image for social sharing previews." Details: A preview of the OG image (icon centered on a complementary solid or gradient background that extends to fill the 1200×630 frame). The background for the OG image is auto-derived from the canvas background, extended/extrapolated to the wider format. Output: Single `og-image.png` file (or included in the Web/PWA zip).

Target 5: "SVG Source Files" Description text: "Clean, optimized vector files for designers and developers." Sub-options (checkboxes, all checked by default):

- "Composed scene (single SVG)" — the entire canvas composition as one SVG file, SVGO-optimized.
- "Individual elements (separate SVGs)" — each element as its own file, SVGO-optimized.
- "Color-tokenized SVG" — the composed scene with colors replaced by CSS custom properties, plus a CSS snippet file. Output: `.zip` containing the selected SVG files + optional `colors.css`.

Target 6: "PNG (Custom Size)" Description text: "Export at any resolution." Input: A numeric field for the export size (default: 1024, range: 64-4096). The aspect ratio is always 1:1 (matching the canvas container). Sub-option: "Transparent background" checkbox (default: unchecked). When checked, the container background is not rendered — only the foreground elements are exported on a transparent background. Output: Single PNG file at the specified resolution.

Modal Footer:

A summary line: "3 packages selected" (updates dynamically).

"Download All" button (primary action). Clicking triggers the export process:

1. The frontend serializes the current canvas state (all element positions, transforms, colors, background settings, container shape) into a `CanvasExportPayload` and sends it to a server-side export endpoint.
2. The server renders the composition using `resvg-js` (for SVG-to-PNG rasterization) and `sharp` (for resizing to specific dimensions).
3. The server assembles the requested packages (generating the correct folder structures, manifests, and meta files).
4. The server returns a download URL for a combined `.zip` containing all selected packages, organized in folders.

The modal shows a progress indicator during export ("Rendering at 1024×1024... Generating iOS sizes... Packaging..."). Export should complete within 5-10 seconds for a typical package.

"Download All" button becomes active only when at least one target is selected.

7.2 Canvas Export Payload

```json
```

```
{
  "session_id": "sess_a1b2c3d4",
  "canvas": {
    "container": {
      "shape": "ios_squircle",
      "corner_radius_percent": null,
      "width": 1024,
      "height": 1024
    },
    "background": {
      "type": "linear_gradient",
      "from": "#F97316",
      "to": "#EC4899",
      "angle": 135
    },
    "elements": [
      {
        "element_id": "el_gen_003",
        "svg_storage_key": "sessions/sess_a1b2c3d4/elements/el_gen_003.svg",
        "transform": {
          "x": 180,
          "y": 140,
          "scale": 0.72,
          "rotation": 0,
          "opacity": 1.0,
          "flip_h": false,
          "flip_v": false
        },
        "color_overrides": {
          "#FFFFFF": "#FFFFFF",
          "#1A1A1A": "#2D2D2D",
          "#F97316": "#FB923C"
        },
        "layer_order": 1,
        "visible": true
      }
    ]
  },
  "export_targets": ["ios_icon_set", "web_pwa_favicons", "svg_source_composed", "svg
}
```


The `color_overrides` map contains ALL detected colors as keys, even if the user didn't change them (key and value will be identical for unchanged colors). This ensures the server can reconstruct the exact visual state without needing to know which colors were "originally" in the SVG.

7.3 iOS Export — Exact Size Table

The `Contents.json` and file list must follow Apple's current specification. Here is the exact set of files to generate:

Filename	Size (px)	Idiom	Scale	Point Size
icon-20@2x.png	40×40	iphone	2x	20pt
icon-20@3x.png	60×60	iphone	3x	20pt
icon-29@2x.png	58×58	iphone	2x	29pt
icon-29@3x.png	87×87	iphone	3x	29pt
icon-40@2x.png	80×80	iphone	2x	40pt
icon-40@3x.png	120×120	iphone	3x	40pt
icon-60@2x.png	120×120	iphone	2x	60pt
icon-60@3x.png	180×180	iphone	3x	60pt
icon-20.png	20×20	ipad	1x	20pt
icon-20@2x.png	40×40	ipad	2x	20pt
icon-29.png	29×29	ipad	1x	29pt
icon-29@2x.png	58×58	ipad	2x	29pt
icon-40.png	40×40	ipad	1x	40pt
icon-40@2x.png	80×80	ipad	2x	40pt
icon-76.png	76×76	ipad	1x	76pt
icon-76@2x.png	152×152	ipad	2x	76pt
icon-83.5@2x.png	167×167	ipad	2x	83.5pt
icon-1024.png	1024×1024	ios-marketing	1x	1024pt
icon-128.png	128×128	mac	1x	128pt
icon-256.png	256×256	mac	1x	256pt
icon-256@2x.png	512×512	mac	2x	256pt
icon-512.png	512×512	mac	1x	512pt
icon-512@2x.png	1024×1024	mac	2x	512pt

The `Contents.json` must reference every file with the correct `idiom`, `size`, and `scale` fields. This file is generated programmatically from the table above — it is not a static template, because

Apple may update the requirements and we need a clear source of truth to modify.

7.4 Android Export — Density Bucket Table

Density	Scale	Launcher Icon Size (px)	Foreground Size (px)
mdpi	1×	48×48	108×108
hdpi	1.5×	72×72	162×162
xhdpi	2×	96×96	216×216
xxhdpi	3×	144×144	324×324
xxxhdpi	4×	192×192	432×432

The foreground layer is rendered at 108dp × scale with the safe zone (inner 66dp circle) containing all critical content. The export pipeline should validate that the user's foreground elements fit within the safe zone and show a warning in the export modal if they don't ("Some elements may be clipped on Android devices with circular icon masks").

7.5 SVGO Optimization Configuration

The SVGO pipeline uses these specific plugins and settings for the "optimized SVG" export:

Plugins to enable: `removeDoctype`, `removeXMLProcInst`, `removeComments`, `removeMetadata`, `removeEditorsNSData`, `cleanupAttrs`, `mergeStyles`, `inlineStyles`, `minifyStyles`, `cleanupIds` (with `minify: true`), `removeUselessDefs`, `cleanupNumericValues` (with `floatPrecision: 2`), `convertColors` (with `shortname: true`), `removeUnknownsAndDefaults`, `removeNonInheritableGroupAttrs`, `removeUselessStrokeAndFill`, `cleanupEnableBackground`, `removeEmptyAttrs`, `removeEmptyContainers`, `convertPathData` (with `floatPrecision: 2`), `convertTransform`, `removeEmptyText`, `sortAttrs`.

Plugins explicitly DISABLED: `removeViewBox` (we need viewBox for scalability), `removeDimensions` (keep width/height for compatibility), `collapseGroups` (we need group structure for the "layered" export variant).

For the "color-tokenized" variant, an additional custom transform runs after SVGO that regex-replaces hex color values with CSS `var()` references. The mapping is generated from the user's color palette: the most-used color becomes `--brand-primary`, the second-most `--brand-secondary`, and so on. A `colors.css` file is generated alongside:

```
css
```

```
:root {
  --brand-primary: #F97316;
  --brand-secondary: #EC4899;
  --brand-accent: #FFFFFF;
  --brand-dark: #1A1A1A;
}
```

---

8. Session Persistence & Retrieval

8.1 Data Model

Every session is persisted as a row in a `sessions` table with the full agent state stored as JSONB. The generated SVG files are stored in object storage (Supabase Storage), not in the database.

```
Table: sessions
─────────────────────────────────────────

id              UUID PRIMARY KEY
user_id         UUID REFERENCES users(id)
title           TEXT DEFAULT 'Untitled Logo'
status          TEXT CHECK (status IN ('intake', 'generating', 'canvas',
'exported'))
intake_data     JSONB
agent_state     JSONB
canvas_state    JSONB
created_at      TIMESTAMPTZ DEFAULT now()
updated_at      TIMESTAMPTZ DEFAULT now()
```

```
Table: session_elements
─────────────────────────────────────────

id              UUID PRIMARY KEY
session_id      UUID REFERENCES sessions(id)
element_id      TEXT UNIQUE
element_type    TEXT
generation_round INTEGER
svg_storage_key TEXT
thumbnail_url   TEXT
detected_colors JSONB
description     TEXT
created_at      TIMESTAMPTZ DEFAULT now()
```

The `canvas_state` JSONB is updated whenever the user makes a canvas change (debounced — save at most once every 2 seconds after the last interaction, plus on page unload via

`beforeunload`). This enables the "retrieve session anytime" requirement — when the user returns to a session, the frontend loads `canvas_state` and reconstructs the canvas exactly as they left it.

8.2 Session List / Dashboard

The dashboard (the landing page for authenticated users) shows a grid of session cards, sorted by `updated_at` descending. Each card shows: the session title, a thumbnail preview of the current canvas state (rendered server-side and cached), the creation date, and the session status (badge: "In Progress" / "Exported"). Clicking a card opens the canvas screen with the full session state restored.

New session action: A prominent "+ New Logo" card at the top of the grid that opens the intake flow (Screen 1).

`MVP_BOUNDARY:` Sessions are per-user, private, and not shareable. Post-MVP, we add shared session links and team collaboration.

---

9. Authentication & User Management

`MVP_BOUNDARY:` MVP uses a minimal auth flow. The user can use the product without authenticating through the intake and first generation (like a "try before you sign up" experience). Authentication is prompted when the user attempts to: save a session, export in any format other than a basic single PNG, or access the dashboard.

Auth is implemented via Supabase Auth with email magic link (no passwords in MVP) and Google OAuth as the only social provider. After authentication, the anonymous session is associated with the new user account.

---

10. API Endpoints

10.1 Endpoint Summary

Method	Path	Purpose	Auth
POST	`/api/sessions`	Create new session from intake payload	Optional (anon allowed)
GET	`/api/sessions`	List user's sessions for dashboard	Required
GET	`/api/sessions/:id`	Get full session state	Required (owner only)

Method	Path	Purpose	Auth
PATCH	`/api/sessions/:id`	Update session title	Required
POST	`/api/sessions/:id/generate`	Trigger initial generation or regeneration	Required
PATCH	`/api/sessions/:id/canvas`	Update canvas state (auto-save)	Required
POST	`/api/sessions/:id/export`	Trigger export pipeline	Required
GET	`/api/sessions/:id/export/:exportId`	Download export zip	Required
GET	`/api/elements/:elementId.svg`	Serve raw SVG for a generated element	Required
GET	`/api/thumbnails/:elementId.png`	Serve PNG thumbnail of element	Required

10.2 Key Endpoint Details

POST `/api/sessions/:id/generate`

Request body for initial generation:

```json
{ "type": "initial" }
```

The intake data is already stored in the session from the `POST /api/sessions` call.

Request body for regeneration:

```json
{
  "type": "regenerate",
  "instruction": "Make the cat simpler, try one with a dog",
  "keep_elements": ["el_gen_003"],
  "discard_elements": ["el_gen_001", "el_gen_002"]
}
```

Response (for both): The `GenerationResponse` object defined in Section 4.6. The endpoint is synchronous (long-polling) — the response is not sent until generation is complete. The client shows the loading screen during this wait. Timeout: 90 seconds.

`MVP_BOUNDARY:` We do NOT use WebSockets or SSE for generation progress in MVP. The loading screen messages are timer-driven on the client. Post-MVP, we can add server-sent events for real-time progress.

POST `/api/sessions/:id/export`

Request body: The `CanvasExportPayload` defined in Section 7.2.

Response:

```json
{
  "export_id": "exp_xyz789",
  "status": "processing",
  "estimated_seconds": 8
}
```

The client polls `GET /api/sessions/:id/export/:exportId` every 2 seconds until `status` becomes `"ready"`, at which point the response includes a `download_url` (signed URL, 1-hour expiry) for the zip file.

---

11. Error Handling & Edge Cases

11.1 Recraft API Failures

If a Recraft generation call fails (timeout, 5xx, rate limit), the agent retries once with a 3-second delay. If the retry also fails, the agent marks that element as `"status": "failed"` in the generation response and continues with the successfully generated elements. The frontend displays the successful elements normally and shows a subtle warning: "1 of 6 elements couldn't be generated. You can try regenerating." The user is never shown a blank canvas due to partial failures.

If ALL generation calls fail, the agent returns an error response and the frontend shows: "We couldn't generate your logo right now. This is usually temporary. [Try again]"

11.2 SVG Parsing Edge Cases

Generated SVGs from Recraft may contain unexpected structures. The following normalization steps must be applied server-side before storing each SVG:

1. Strip any `<image>` elements that embed raster data (fallback behavior some SVG generators use). If an SVG contains only raster data and no paths, log a warning and mark the element as `"status": "quality_warning"`.
2. Normalize all color values to 6-digit hex format (convert `rgb()`, `hsl()`, shorthand `#FFF`, named colors).

3. Ensure a `viewBox` attribute exists on the root `<svg>` element. If missing, compute it from the content bounds.
4. Remove any embedded `<script>` or `<foreignObject>` elements (security).
5. Remove XML processing instructions and DOCTYPE declarations.

11.3 Canvas Auto-Save Failures

If a canvas state save fails (network error), the frontend stores the state in memory and retries on the next debounce cycle. If 3 consecutive saves fail, show a non-blocking toast: "Changes aren't saving. Check your connection." The canvas remains fully functional — data loss only occurs if the user closes the tab while offline (an accepted MVP limitation).

11.4 Export Rendering Failures

If the server-side SVG-to-PNG rendering fails (malformed SVG, memory limits), the export endpoint returns an error for the affected target. The frontend shows: "iOS export failed — the SVG may contain unsupported features. Try simplifying your design or removing a complex element." Other targets in the same export batch that succeeded are still available for download.

---

12. Performance Budgets

These are hard targets, not aspirations. If any of these are exceeded, it is a bug.

Metric	Target
Intake form load (LCP)	< 1.5s
Canvas screen load (with session restore)	< 2.5s
Canvas interaction latency (drag, resize, rotate)	< 16ms (60fps)
Element recolor (applying new color across SVG)	< 100ms
Canvas auto-save (debounced write to server)	< 500ms
Export rendering (single 1024×1024 PNG)	< 3s
Full export package (iOS + Android + Web + SVGs)	< 15s
Generation round (agent + Recraft calls)	< 30s typical, < 60s worst case

The canvas interaction budget (16ms) means the SVG DOM manipulation for drag/resize must NOT cause layout thrashing. Element transforms should be applied via the `transform` attribute

on a wrapping `<g>` element, not by modifying individual path coordinates. The transform attribute is GPU-accelerated in modern browsers and does not trigger reflow.

---

13. MVP Boundaries — What We Are Explicitly NOT Building

This list exists to prevent scope creep during development. Each item is a conscious exclusion, not an oversight.

Feature	Status	Rationale
Icon pack generation (matching UI icons)	Post-MVP (V1.2)	Depends on validating the logo flow first
Mascot pose variations	Post-MVP (V1.3)	Requires additional Recraft prompt engineering
Text tool on canvas	Post-MVP (V1.1)	Adds significant complexity to the canvas; users can add text in Figma after export
Figma plugin integration	Post-MVP (V2)	Requires Figma API integration and a separate codebase
React component export (.tsx)	Post-MVP (V2)	Requires SVGR integration and TypeScript generation
Team workspaces / collaboration	Post-MVP (V2)	Requires access control, real-time sync
Multi-select on canvas	Post-MVP	Adds complexity to transform calculations
Drag from gallery to canvas	Post-MVP	Click-to-add is sufficient for MVP
Mobile canvas editing	Post-MVP	Desktop-first; mobile shows read-only preview
Style presets / templates	Post-MVP	Let the AI generate from scratch first
Custom font upload	Post-MVP	Not needed without text tool
Animation export (Lottie)	Post-MVP (V3+)	Different rendering pipeline entirely
Per-element regeneration	Post-MVP	MVP uses batch regeneration only
Streaming generation progress	Post-MVP	Timer-driven loading screen is sufficient
Dark mode for the application UI	V1.1	Standard post-launch polish

14. Quality Checklist Before Ship

Every item below must be true before the MVP goes live. This is not a testing plan — it is a product acceptance criteria list.

The intake flow completes in under 90 seconds for a user who knows what they want. The style grid images are high-quality and genuinely representative of each cluster. The personality word pills are responsive and feel snappy on tap. The loading screen never shows a broken state — even if generation takes 60 seconds, the experience feels intentional. Generated SVG elements are visually clean and match the selected style cluster at least 80% of the time (subjective but calibrated through internal testing). The canvas loads with elements already composed intelligently — not dumped in a pile. Drag, resize, and rotate feel native — no jank, no lag, no unexpected jumps. The recolor panel detects and displays the correct set of colors for every generated SVG. Alignment guides appear at the right moments and snapping feels helpful, not frustrating. The export modal clearly communicates what the user will receive. Every export target produces files that work out of the box — the iOS set opens in Xcode without modification, the Android set compiles in Android Studio, the web favicons render correctly in Chrome/Safari/Firefox. The SVGO-optimized SVGs open cleanly in Figma and Illustrator. Session restoration is pixel-perfect — returning to a session looks exactly like you left it. The product handles partial generation failures gracefully — the user always sees something, never a blank screen.

This document is the complete specification for the VectorShip MVP. It defines what we ship, how it behaves, and where we draw the line. Everything within this scope should be built with production-quality attention to detail. Everything outside this scope should be deferred without guilt.